SecureLab: Video Demonstration Scripts & Viva Guide

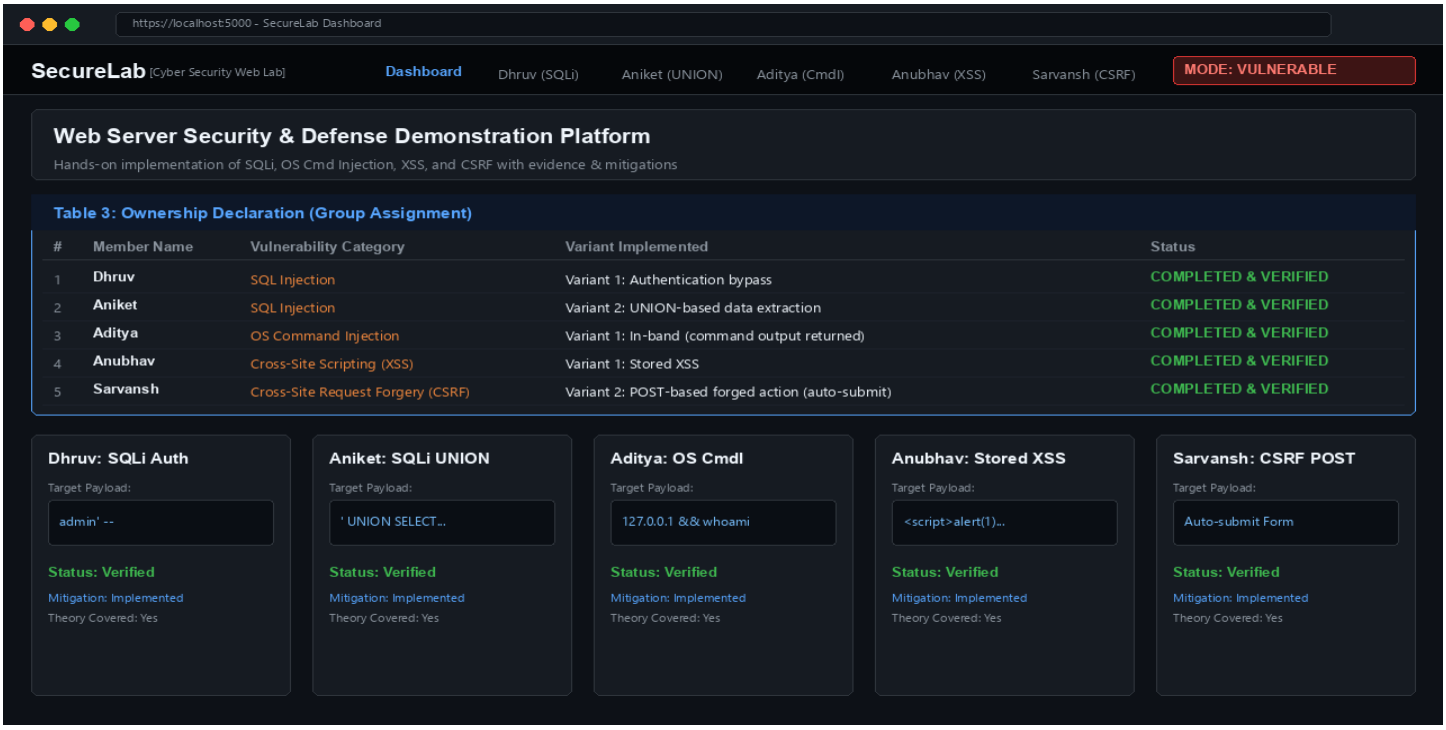
Hands-on Web Server Security: SQLi, OS Command Injection, XSS, and CSRF

Group Members: Dhruv, Aniket, Aditya, Anubhav, Sarvansh

Table 3: Ownership Declaration (Course Compliance)

#	Member Name	Category	Variant Implemented	Timing
1	**Dhruv**	SQL Injection	Variant 1: Authentication bypass	2.5 mins (0:00 - 2:30)
2	**Aniket**	SQL Injection	Variant 2: UNION-based data extraction	2.5 mins (2:30 - 5:00)
3	**Aditya**	OS Command Injection	Variant 1: In-band (command output returned)	2.5 mins (5:00 - 7:30)
4	**Anubhav**	Cross-Site Scripting (XSS)	Variant 1: Stored XSS	2.5 mins (7:30 - 10:00)
5	**Sarvansh**	CSRF	Variant 2: POST-based forged action (auto-submit)	2.5 mins (10:00 - 12:30)

Figure 1: SecureLab Interactive Security Dashboard & Table 3 Live Portal



Member 1: Dhruv — SQL Injection (Auth Bypass)

Target Variant: SQL Injection — Variant 1 (Authentication Bypass) • **Time:** ~2.5 Minutes

Step-by-Step Spoken Script & Actions (Video Voice-Over):

ACTION 1: [Screen shows SecureLab Dashboard, mouse points to Dhruv's row in Table 3, clicks 'Demo Auth Bypass']

"Hello everyone. My name is Dhruv, and I am demonstrating Variant 1 of SQL Injection: Authentication Bypass. As seen on our project dashboard, I personally implemented and secured this module on target endpoint POST /sqli-auth."

ACTION 2: [Ensure the navbar displays 'VULNERABLE MODE'. In the Username field, type admin' --. In Password, type anyPassword123. Click 'Attempt Authentication'.]

"First, I will demonstrate the flaw under our application's Vulnerable Mode. I enter admin' -- in the username field, and provide any random password. When I click Login, notice the red alert: we have successfully bypassed authentication and logged in as the administrator without knowing the password!"

ACTION 3: [Scroll down to the server-executed query box showing the raw SQL statement.]

*"The root cause is unvalidated string interpolation. The server formats raw user input into: SELECT * FROM users WHERE username = 'admin' --' AND password = '...'. The single quote prematurely terminates the username string, and the double-dash instructs the SQLite engine to treat everything following it as a comment, completely short-circuiting the password verification logic."*

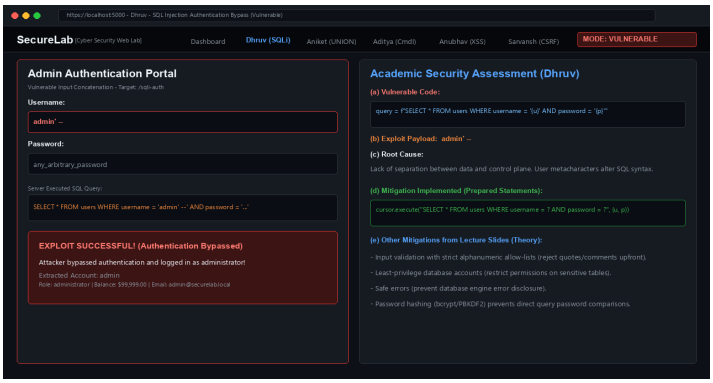
ACTION 4: [Click the top navbar button 'Click to Secure'. Show the green banner 'SECURE / MITIGATED MODE'. Type the exact same payload admin' -- and click Login.]

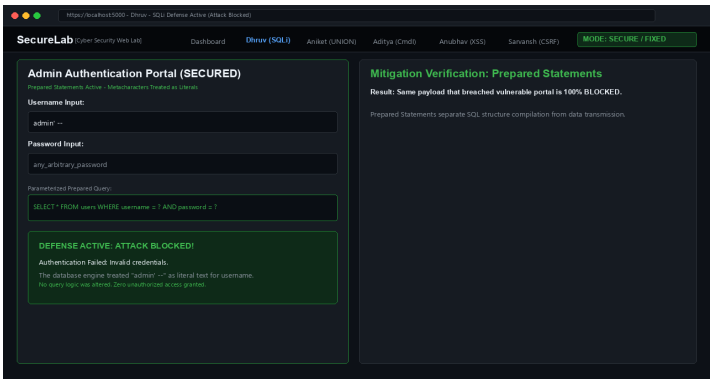
*"Now, I activate our implemented defense by toggling the platform into Secure Mode. I submit the exact same exploit payload: admin' --. Notice that authentication is blocked with 'Invalid credentials'. Our mitigation is **Parameterized Queries with Prepared Statements** using positional placeholders. The database pre-compiles the query tree, and treats admin' -- purely as a literal string value, not executable SQL syntax."*

ACTION 5: [Summarize other lecture mitigations for 20 seconds to conclude.]

"Beyond prepared statements, other mitigations discussed in class include strict alphanumeric allow-lists to reject quotes upfront, least-privilege database user accounts, suppressing error messages, and storing salted password hashes using bcrypt or PBKDF2. I now hand over to Aniket for Variant 2."

Evidence 1 (Left: Exploit Working • Right: Attack Blocked by Prepared Statements):





Member 2: Aniket — SQL Injection (UNION Data Extraction)

Target Variant: SQL Injection — Variant 2 (UNION-Based Data Extraction) • **Time:** ~2.5 Minutes

Step-by-Step Spoken Script & Actions (Video Voice-Over):

ACTION 1: [Screen navigates to /sqli-union (Product Catalog Search), showing Table 3 row for Aniket.]

"Hello everyone. My name is Aniket, and I am presenting Variant 2 of SQL Injection: UNION-Based Data Extraction. While Dhruv bypassed authentication logic, my implementation demonstrates how an attacker extracts unauthorized database records into public tables."

ACTION 2: [Ensure Vulnerable Mode is active. Enter normal search 'Camera' first, then paste payload: ' UNION SELECT id, username, password, role FROM users -- and click Search.]

"In our vulnerable implementation, the search parameter is concatenated directly into the product query's LIKE clause. A normal search returns product appliances. But when I inject: ' UNION SELECT id, username, password, role FROM users -- and click Search, look at the results table: the database executes both queries and appends secret user records, passwords, and administrative roles directly into the public product columns!"

ACTION 3: [Highlight the executed query box and explain root cause.]

"The root cause is that SQLite allows the set operator UNION to combine results from two queries having matching column counts (4 columns here). Because user input was not parameterized, the attacker forces the database to disclose confidential internal tables."

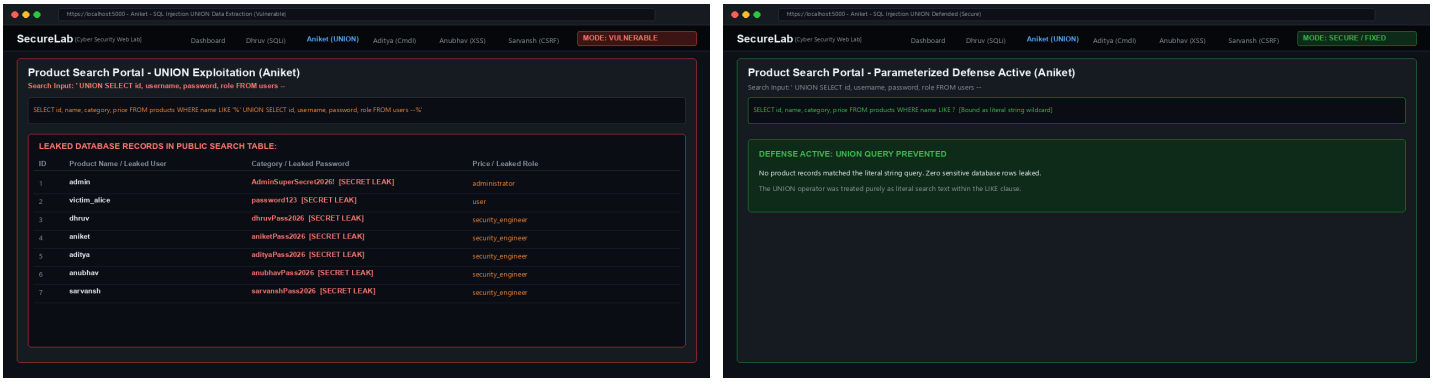
ACTION 4: [Toggle system to Secure Mode. Submit the exact same UNION payload in search box.]

*"To secure this endpoint, I implemented **Parameterized Wildcard Binding** using cursor.execute('... WHERE name LIKE ?', (f'%{q}%',)). When I run the exact same UNION attack in Secure Mode, the database treats the entire payload as literal text. Zero sensitive rows are leaked, and the search simply reports 'No matching records returned'."*

ACTION 5: [Summarize lecture mitigations.]

"Additional mitigations from our lecture include configuring least-privilege database user permissions—so the web search user has zero SELECT access on the users table—using Object-Relational Mappers (ORMs), and suppressing column mismatch errors. I now pass over to Aditya for OS Command Injection."

Evidence 2 (Left: Leaked Passwords via UNION • Right: Attack Neutralized by Parameterized Query):



Member 3: Aditya — OS Command Injection (In-Band)

Target Variant: OS Command Injection — Variant 1 (In-Band Output Returned) • **Time:** ~2.5 Minutes

Step-by-Step Spoken Script & Actions (Video Voice-Over):

ACTION 1: [Screen navigates to /cmdi-inband (Network Diagnostic Utility / Ping Tool).]

"Hello, I am Aditya. I am demonstrating Variant 1 of OS Command Injection: In-Band execution, on target endpoint POST /cmdi-inband. This category occurs when a web application executes operating system commands on the server using unsanitized user inputs."

ACTION 2: [Ensure Vulnerable Mode is active. Enter 127.0.0.1 && whoami in the target host field and click Execute Ping.]

*"In Vulnerable Mode, our diagnostic utility takes an IP address and executes ping -n 1 {target} with Python's subprocess.run(cmd, shell=True). I submit the payload 127.0.0.1 && whoami. As you can see in the in-band terminal output, after pinging localhost, the server immediately executes whoami and prints the operating system user account directly onto our webpage! This is an **in-band exploit** because the command output is returned in the application's normal HTTP response channel."*

ACTION 3: [Explain root cause clearly.]

"The root cause is invoking a system shell with shell=True. The shell interpreter evaluates characters like &&, &, ;, and | as command chaining separators, allowing arbitrary unauthorized commands to run with the privileges of the web server process."

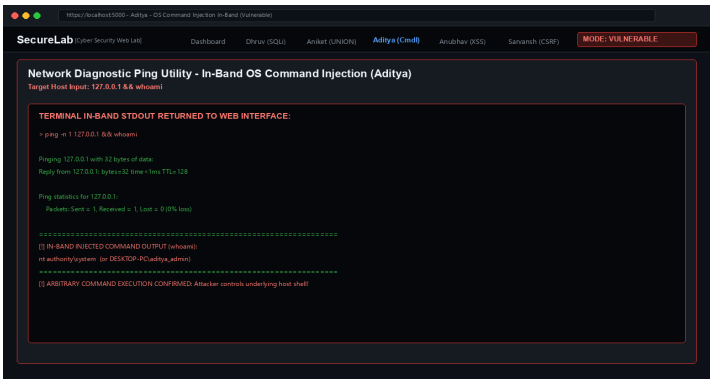
ACTION 4: [Toggle platform to Secure Mode. Submit the exact same payload: 127.0.0.1 && whoami.]

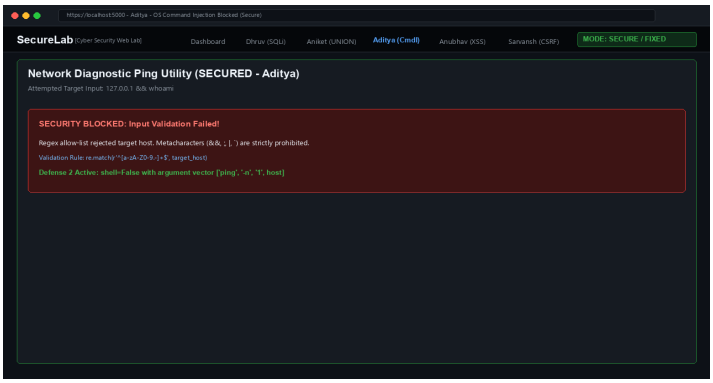
*"Now I switch to Secure Mode. I run the exact same payload: 127.0.0.1 && whoami. Notice the red security banner: **SECURITY BLOCKED**. My implemented mitigation employs two layers of defense: First, **strict regex allow-list validation** using ^[a-zA-Z0-9.-]+$ which instantly rejects all shell metacharacters. Second, **avoiding the shell** by setting shell=False and passing an argument vector array ['ping', '-n', '1', target] directly to the OS kernel."*

ACTION 5: [Summarize lecture mitigations.]

"Other mitigations from class include avoiding system binary execution altogether by using native language APIs like Python's socket library, and running web processes under heavily restricted, unprivileged service accounts or Docker containers. Next is Anubhav with Stored XSS."

Evidence 3 (Left: In-Band Command Output Returned • Right: Blocked by Allow-List & Argument Vector):





Member 4: Anubhav — Cross-Site Scripting (Stored XSS)

Target Variant: Cross-Site Scripting (XSS) — Variant 1 (Stored XSS) • **Time:** ~2.5 Minutes

Step-by-Step Spoken Script & Actions (Video Voice-Over):

ACTION 1: [Screen navigates to /xss-stored (Community Feedback Board).]

"Hello everyone. My name is Anubhav, and I am demonstrating Variant 1 of Cross-Site Scripting: Stored XSS, on target endpoint POST /xss-stored. Stored XSS is the most dangerous form of XSS because the malicious script is persisted in the server database and delivered to every visitor."

ACTION 2: [Ensure Vulnerable Mode is active. In comment field, enter: <script>alert('Stored XSS Executed by Anubhav! Stolen Cookie: ' + document.cookie);</script> and click Submit.]

"In Vulnerable Mode, user feedback is rendered using Jinja2's | safe filter, which disables HTML escaping. I submit this payload containing a script tag that reads document.cookie. Upon submission—and whenever any user visits or refreshes this page—the browser automatically parses and executes the attacker's script, displaying an alert box with the victim's session cookie! An attacker could transmit this cookie to an external server to hijack the session."

ACTION 3: [Explain root cause.]

"The root cause is rendering untrusted user-controlled data into HTML without context-aware output encoding. The browser cannot distinguish between legitimate scripts written by the developer and malicious scripts injected by users."

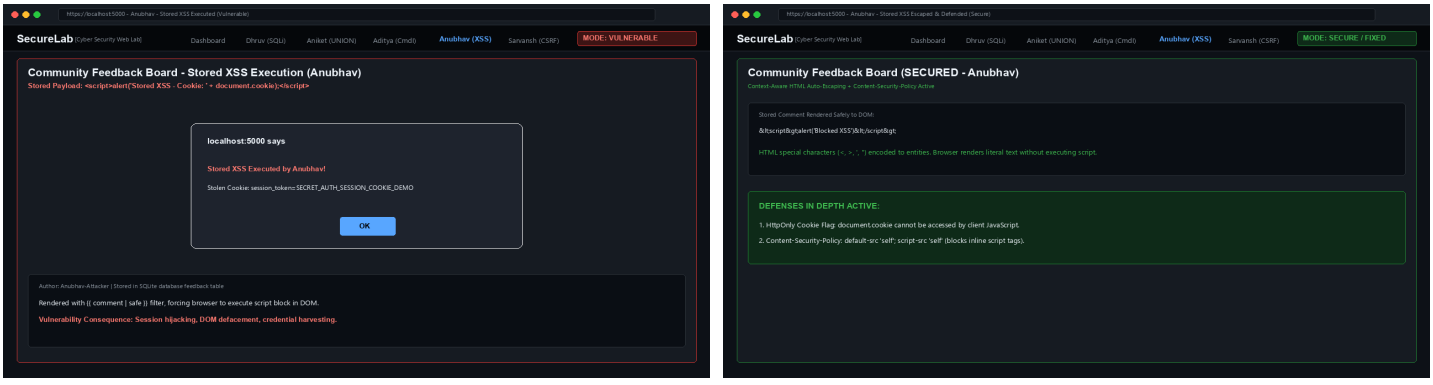
ACTION 4: [Toggle to Secure Mode. Submit the exact same XSS script payload. Show safe rendering.]

*"Now I activate Secure Mode. I submit the exact same script payload. Notice that no alert pop-up appears! Instead, the text is safely displayed as literal characters: <script>. Our implemented mitigation removes the | safe filter, enforcing **Context-Aware HTML Entity Auto-Escaping**. Furthermore, our server sets the HttpOnly cookie flag and enforces a **Content Security Policy (CSP)** header blocking inline scripts."*

ACTION 5: [Summarize other lecture mitigations.]

"Other mitigations covered in the lecture include rich HTML sanitizers like DOMPurify or Bleach for formatted text, and utilizing safe DOM sinks like textContent instead of dangerous sinks like innerHTML or eval(). Finally, Sarvansh will demonstrate CSRF."

Evidence 4 (Left: Stored Script Executing with Cookie • Right: Defended via HTML Escaping & CSP):



Member 5: Sarvansh — CSRF (POST-Based Auto-Submitting Form)

Target Variant: CSRF — Variant 2 (POST-Based Forged Action via Auto-Submitting Form) • **Time:** ~2.5 Minutes

Step-by-Step Spoken Script & Actions (Video Voice-Over):

ACTION 1: [Screen shows /csrf-post with logged-in user victim_alice and email alice@company.org.]

"Hello everyone. My name is Sarvansh, and I am presenting Variant 2 of Cross-Site Request Forgery (CSRF): POST-based forged action with an auto-submitting form. CSRF exploits the ambient trust a web application places in a user's browser session."

ACTION 2: [Ensure Vulnerable Mode is active. Open a new browser tab to /attacker-site (simulating an external phishing page).]

"Notice Alice is logged into SecureLab with email alice@company.org. In Vulnerable Mode, the server's update endpoint does not verify CSRF tokens. Now, Alice visits an external hostile website hosted on a simulated attacker domain—a fake prize page at /attacker-site. This page contains a hidden HTML form targeting SecureLab with an auto-submitting JavaScript function: document.getElementById('csrfForm').submit()."

ACTION 3: [The hostile page auto-submits. Redirect back to /csrf-post. Show email changed to attacker_sarvansh@hacker.io.]

"Because Alice's browser has an active session, it automatically attaches her session cookies to the cross-site POST request. Look at Alice's profile now: her email has been compromised and changed to attacker_sarvansh@hacker.io! The root cause is that the server processed a state-changing request without proof of user intent, blindly trusting ambient cookie authentication."

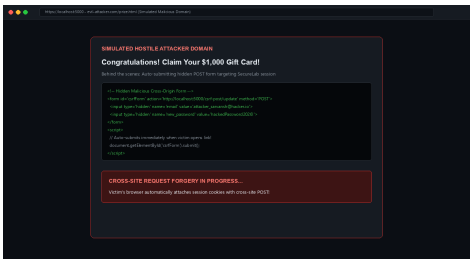
ACTION 4: [Toggle system to Secure Mode. Trigger the hostile attacker page again. Show HTTP 403 Forbidden.]

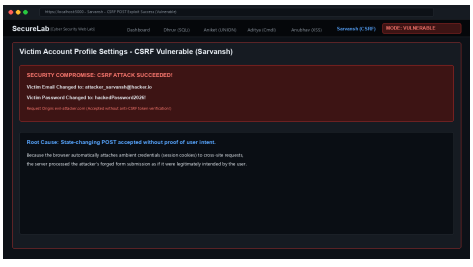
*"To defend against this, I implemented the **Synchronizer Token Pattern** and **SameSite Cookies**. The server issues a cryptographically random, per-session csrf_token embedded only in legitimate forms. When the attacker's external page attempts the exact same cross-origin POST, the request is rejected with **HTTP 403 Forbidden: Forged CSRF Request Blocked**! Because of the browser's Same-Origin Policy (SOP), the hostile site cannot read Alice's token, making forgery impossible."*

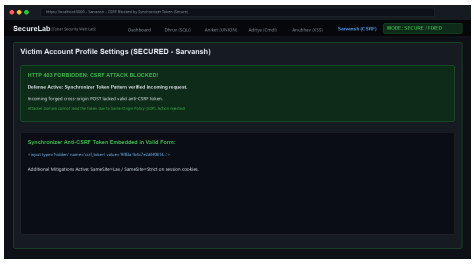
ACTION 5: [Summarize lecture mitigations and conclude.]

"Other mitigations discussed in the lecture include configuring SameSite=Strict or Lax on session cookies, requiring re-authentication (current password/2FA) for sensitive actions, and avoiding state-changing GET requests. This concludes our group's video demonstration covering all four categories."

Evidence 5 (Attacker Hostile Page • Exploit Success • Blocked by Synchronizer Token):







Viva Voce Defense Guide & Expected Questions

Be ready to answer the following questions individually during the oral viva:

Dhruv (SQLi Auth Bypass)

Q: Why does a prepared statement physically prevent SQL injection?

A: In a prepared statement, the database compiles the query execution blueprint before any user data is bound. Parameters are passed out-of-band and inserted strictly as literal data nodes, meaning user metacharacters (quotes, dashes) can never alter the syntax tree or logic operators.

Aniket (SQLi UNION Extraction)

Q: What conditions must be satisfied for a UNION-based attack to succeed?

A: Two conditions: (1) The injected query must return the exact same number of columns as the original query (4 columns in our case). (2) The data types of each corresponding column must be compatible with the original columns.

Aditya (OS Command Injection)

Q: Why is shell=False safer than shell=True in Python's subprocess?

A: shell=True passes the entire command string to cmd.exe or /bin/sh, which parses metacharacters (&&, |, ;) as command delimiters. With shell=False, Python invokes CreateProcess or execve directly with an argument vector array, so arguments cannot spawn new command processes.

Anubhav (Stored XSS)

Q: If HttpOnly is enabled on cookies, does that completely fix XSS?

A: No. HttpOnly only protects session cookies from being read via document.cookie. An attacker with XSS execution can still perform arbitrary actions within the DOM, deface pages, log keystrokes, and initiate authenticated requests on behalf of the victim. Context-aware output encoding and CSP are mandatory.

Sarvansh (CSRF POST Auto-Submit)

Q: Why can an external attacker site not simply steal the Anti-CSRF token?

A: Because of the browser's Same-Origin Policy (SOP). Scripts on external hostile domains (e.g. evil.com) are strictly prohibited from reading HTTP responses or DOM state from another origin (localhost:5000), unless explicitly permitted by CORS headers. Since the attacker cannot read the token, they cannot forge a valid request.